
DSCI 551 COURSE PROJECT, SPRING'26
Phase 3: Demo and Implementation



Hybrid AML Fraud Detection in PostgreSQL + Apache AGE

Group No: DB5

Group Members: Chanyoung Kim, Yogita Mutyala

Problem Statement

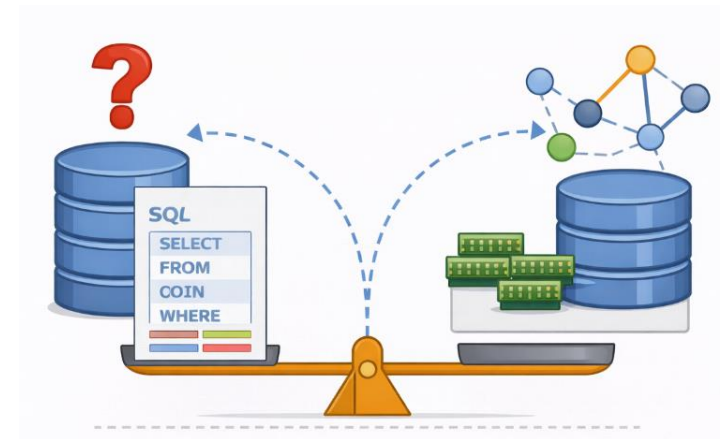
Anti-Money Laundering (AML) data starts as transactional rows, but suspicious behavior appears across multiple accounts and transfers. Moreover, a simple reporting-only view can miss how money moves through networks of related entities.

Research Question

Can PostgreSQL with Apache AGE efficiently support graph traversal workloads compared to native relational approaches, and what are the storage and execution tradeoffs of this hybrid model?

Approach Motivation

- AML patterns span linked accounts and transactions
- Graph model relationships directly (nodes + edges)
- Enables path and cycle analysis within the same PostgreSQL system

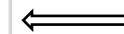


Relational Workload (SQL)

- Joins
- Aggregations
- Filters

Graph Workload (Cypher)

- Paths
- Rings
- Fan-in/out
- Linked Entities



Chosen Database System: PostgreSQL + Apache AGE

What AML needs

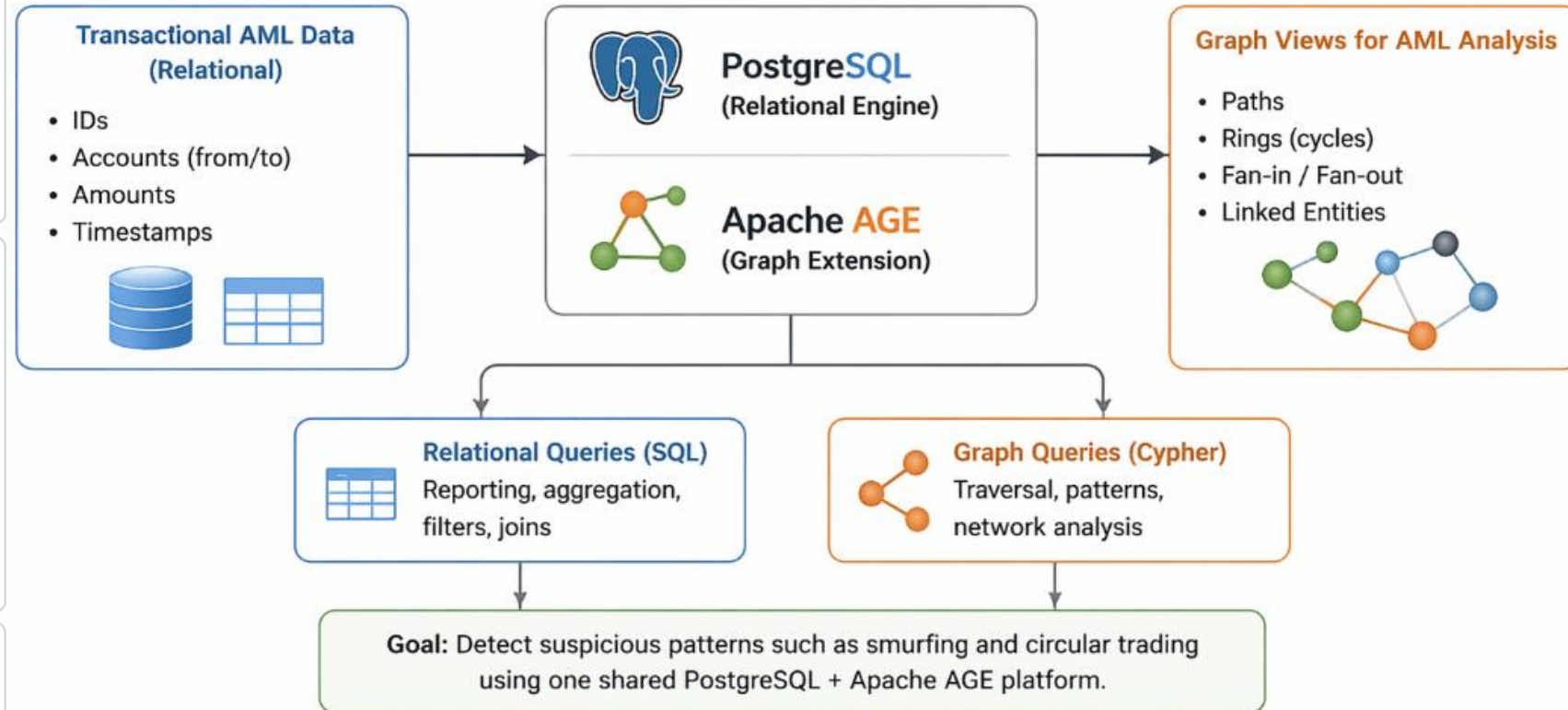
- Transactional logs and customer/account data need ACID relational storage.
- Suspicious patterns such as circular trading require graph-style traversal.
- The demo also needs reproducible loading, filtering, grouping, and benchmarking.

What PostgreSQL gives

- Heap-organized relational tables remain the authoritative store for customers, accounts, and transactions.
- Typed columns, indexes, joins, and recursive SQL make the baseline measurable and practical.
- The same engine handles setup scripts and query execution.

What AGE adds

- Apache AGE brings openCypher into PostgreSQL without moving data to a separate native graph engine.
- Account vertices and TRANSFER edges can be projected from the same relational rows.
- PostgreSQL + AGE avoids maintaining separate relational and graph systems.



Note: Other RDBMS do not natively support graph queries, and separate graph systems add complexity; PostgreSQL + AGE provides ACID guarantees and integrated graph functionality for controlled, fair evaluation without cross-system overhead

About the AML Data

Data Overview:

- 1896 transaction records
- 369 accounts
- 324 customers

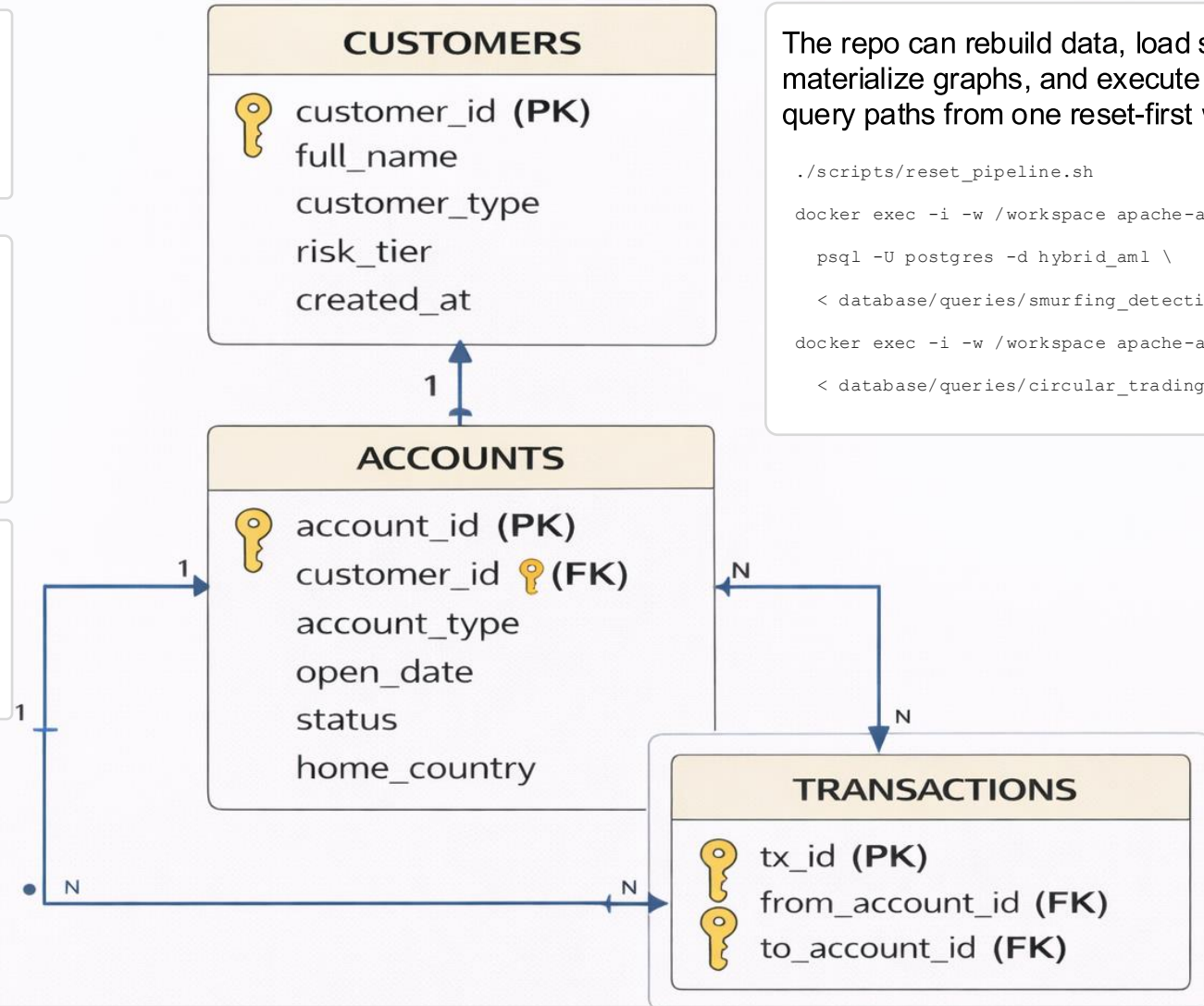
Data Generation

- Synthetic dataset generated using a data generation script
- Enables controlled scenarios (normal, smurfing, circular trading)
- Supports repeatable benchmarking

Key Tables

- Customers -> identity + risk tier
- Accounts -> ownership + account metadata
- Transactions -> money flow + fraud tables

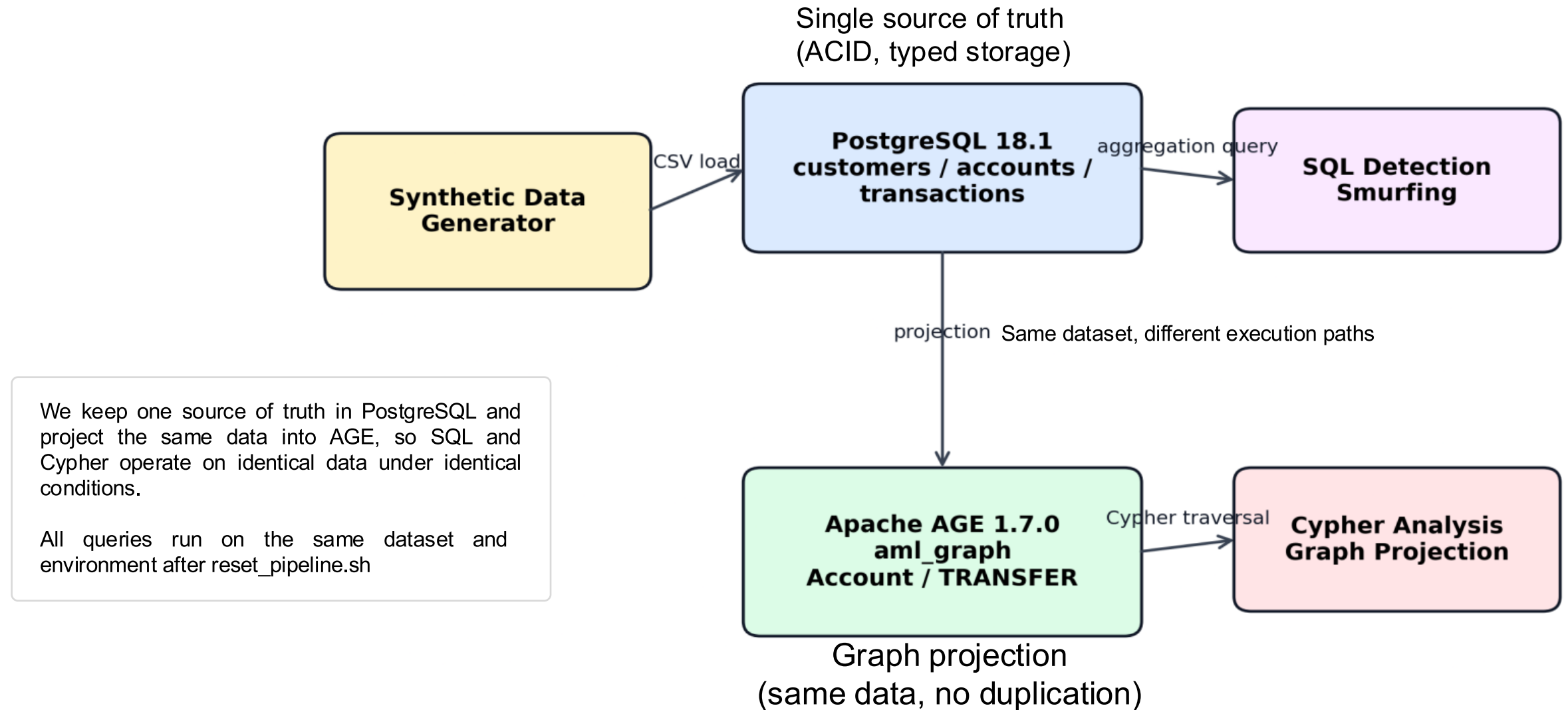
Note: Same dataset is used for both SQL and graph queries to ensure fair comparison



The repo can rebuild data, load schema, materialize graphs, and execute both query paths from one reset-first workflow

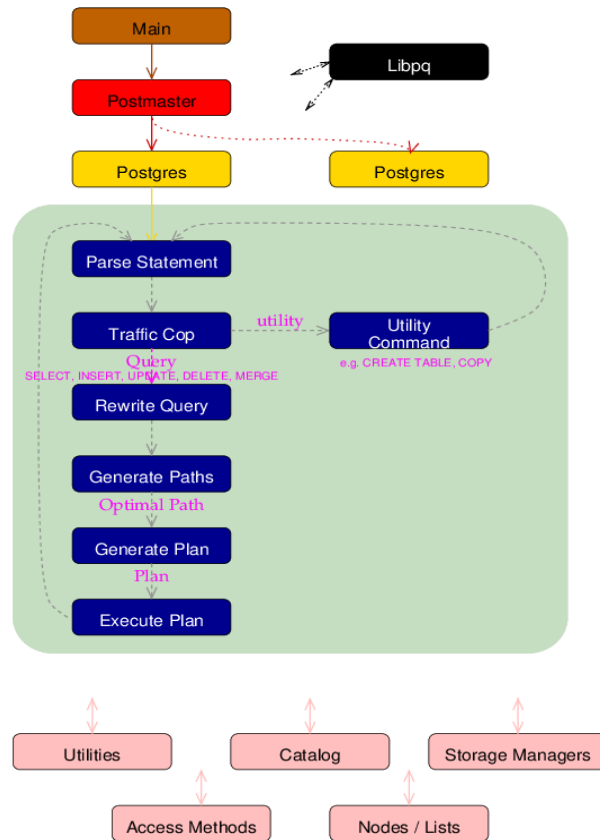
```
./scripts/reset_pipeline.sh  
  
docker exec -i -w /workspace apache-age \  
    psql -U postgres -d hybrid_aml \  
    < database/queries/smurfing_detection.sql  
  
docker exec -i -w /workspace apache-age \  
    < database/queries/circular_trading_cypher_tuned.sql
```

AML Application workflow: Prototype Architecture



Focus Area 1: Storage Architecture

PostgreSQL System Architecture



1. Query Parsing

PostgreSQL first parses incoming SQL and converts it into an internal query representation.

2. Query Rewrite / Planning

The system rewrites complex queries when needed and chooses the lowest-cost execution plan

3. Execution on Heap Storage

The executor runs the selected plan on heap-organized tables, with indexes pointing to the target heap tuples.

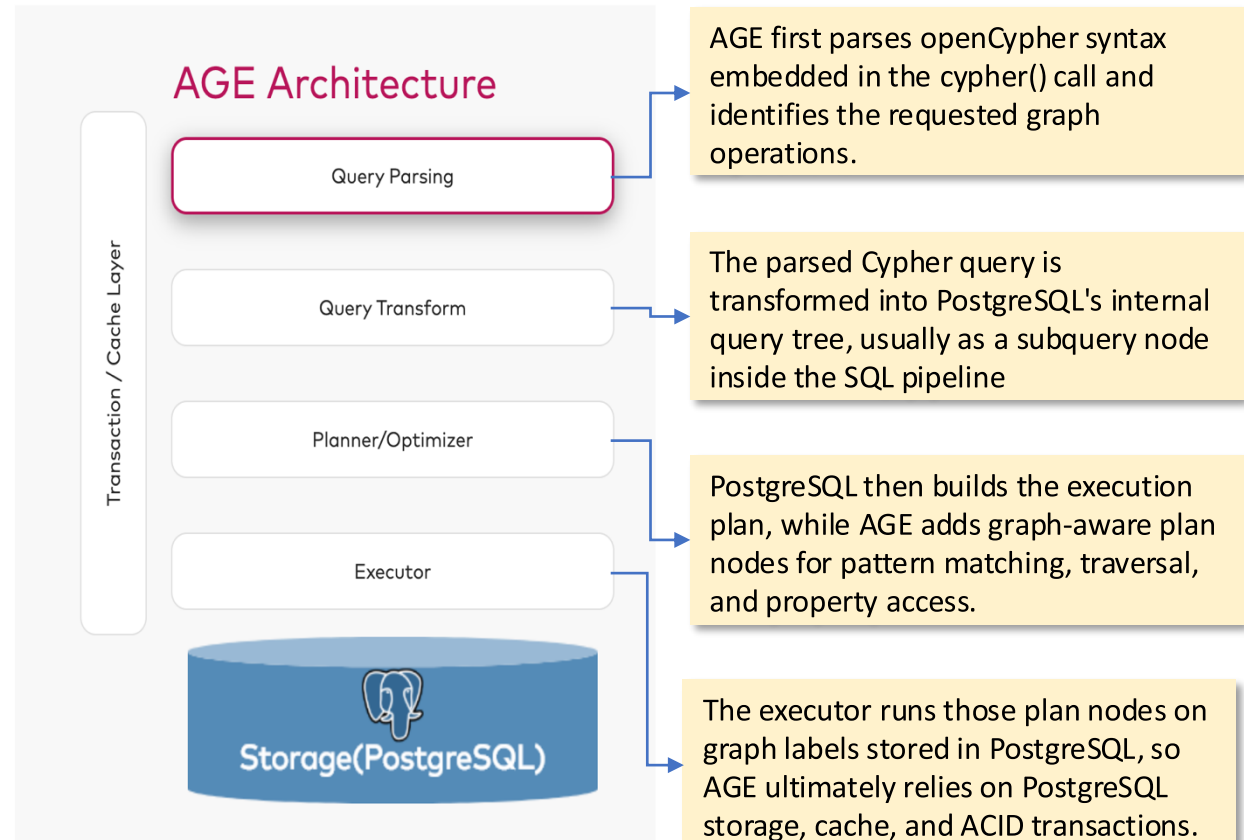
4. MVCC and Cleanup

Concurrency is handled through tuple-version MVCC, and obsolete row versions are later reclaimed by VACUUM.

Difference from other RDBMS

Unlike MySQL with the InnoDB storage engine, PostgreSQL does not separate the SQL layer from a standalone storage engine. It uses a more unified architecture with tuple-version MVCC and stronger extensibility.

APACHE AGE System Architecture

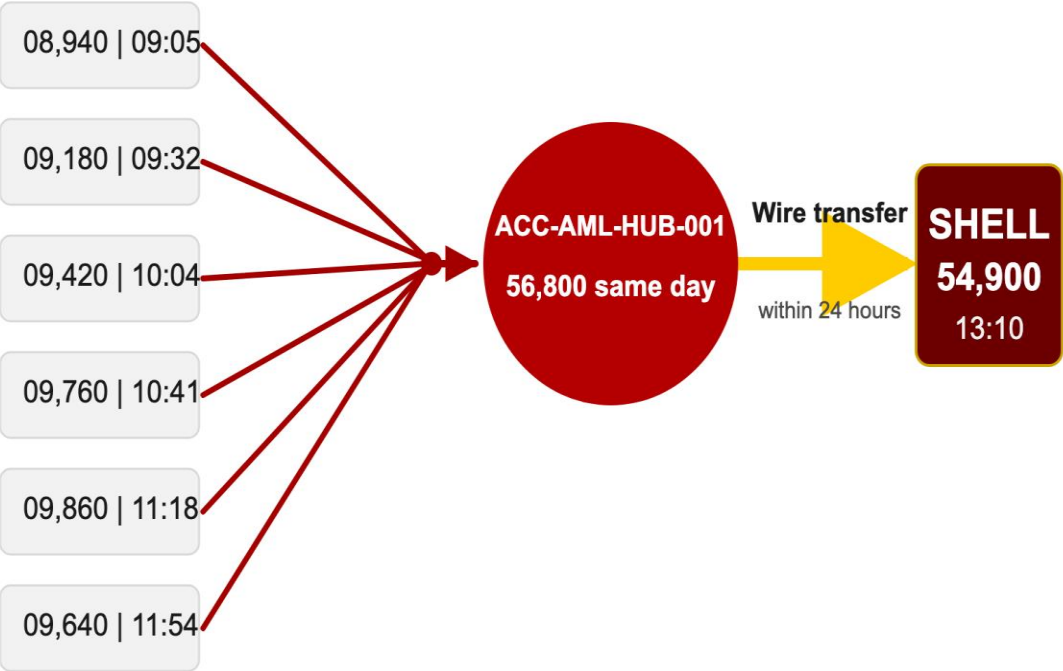


Difference from other Graph DBMS

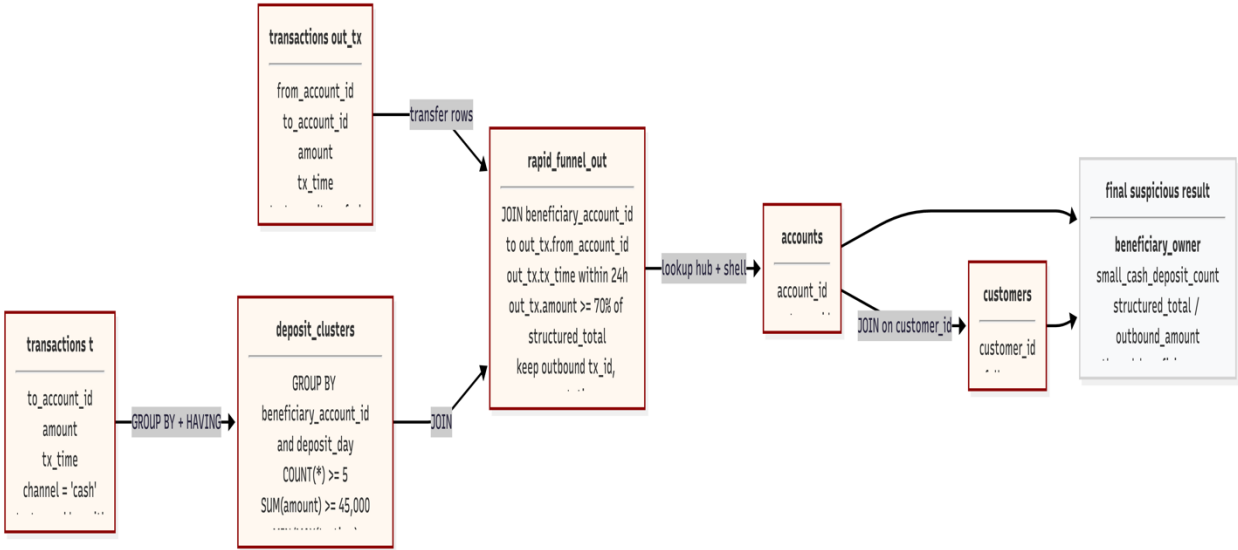
Unlike native graph databases, Apache AGE does not use a separate graph storage engine. It layers graph querying on top of PostgreSQL so relational and graph workloads share one transactional system.

What the application does 1: Smurfing detection

Smurfing Scenario Visual Explanation



SQL Logic



Real-World Example

Several small cash deposits were made into account **ACC-AML-HUB-001** on the same day, including **\$8,940 at 09:05**, **\$9,180 at 09:32**, **\$9,420 at 10:04**, **\$9,760 at 10:41**, **\$9,860 at 11:18**, and **\$9,640 at 11:54**. Although each deposit was made in a relatively small amount, together they totaled **\$56,800 in one day**. Shortly afterward, the account sent a **wire transfer of \$54,900 to SHELL at 13:10**. This pattern is suspicious because multiple sub-threshold deposits were quickly consolidated into one account and then transferred out within a short period of time, which is a common sign of **smurfing or structuring in money laundering**.

Query Explanation 1: Smurfing EXPLAIN analysis

PostgreSQL SQL Execution EXPLAIN

1. Seq Scan on public.transactions

Filter: tx_type='deposit' AND channel='cash'
cost=0.59.44 rows=105 Buffers=31
picks out small cash deposits

2. Sort → GroupAggregate on public.transactions

Keys: to_account_id, date_trunc(day, tx_time)
count(*) / sum(amount) / min / max
HAVING count≥5 AND sum≥45,000 rows=9 / 60

3. Index Scan on public.transactions

idx_transactions_from_account_time
Cond: from_account_id = deposit-beneficiary AND
tx_time within 24h funnel window
Filter: tx_type='transfer' AND amount≥0.70·Σ

4. Nested Loop × 4 on public.accounts & public.customers

accounts_pkey → customers_pkey (Inner Unique)
beneficiary account + customer,
shell account + customer
cost=60.96..69.46 Buffers=169

5. Sort (final)

Sort Key: sum(amount) DESC, out_tx.tx_time
rows=9 Execution Time ≈ 0.443 ms

What the database does internally

Seq Scan on public.transactions picks **small cash deposits** (tx_type='deposit', channel='cash'). **Sort → GroupAggregate** by **beneficiary account + day**, keeping **clusters ≥ 5 deposits** totalling **≥ \$45,000**. For each cluster, **Index Scan** on idx_transactions_from_account_time finds an **outbound wire** within a **24-hour funnel window** carrying **≥ 70% of the deposited sum**. **Four Nested Loop joins** with public.accounts + public.customers add customer names (beneficiary + shell). **Final Sort** by structured_total. Result: **9 hits in ≈ 0.443 ms / 169 shared buffers**.

APACHE AGE Execution EXPLAIN

1. Seq Scan on aml_graph."TRANSFER"

Filter: agtype tx_type='deposit' AND channel='cash'
cost=0.93.92 rows=105 Buffers=56
deposit edges only

2. Index Only Scan on aml_graph."Account"

Account_pkey Cond: id = TRANSFER.start_id
Inner Unique loops=105 Heap Fetches=105
Buffers=210

3. Sort → GroupAggregate on aml_graph."TRANSFER"

agtype_access_operator extracts amount / tx_time
grouped by Account.account_id, day
HAVING count≥5 AND sum≥45,000 rows=9 Buffers=518

4. Hash Join on aml_graph."Account" ⋈ deposit groups

Seq Scan aml_graph."Account" (369 rows, 474 kB)
Hash Cond: Account.account_id = beneficiary_id

5. Index Scan on aml_graph."TRANSFER" (outbound)

TRANSFER_start_id_idx Cond: start_id = Account.id
Nested Loop Filter: time window AND amt≥0.70·Σ
then Index Scan aml_graph."Account" (dst)

6. Nested Loop × 3 on public.accounts & public.customers

accounts_pkey → customers_pkey (agtype → text cast
on every edge) for beneficiary and shell sides
loops=9 Buffers=629..683

7. Sort (final)

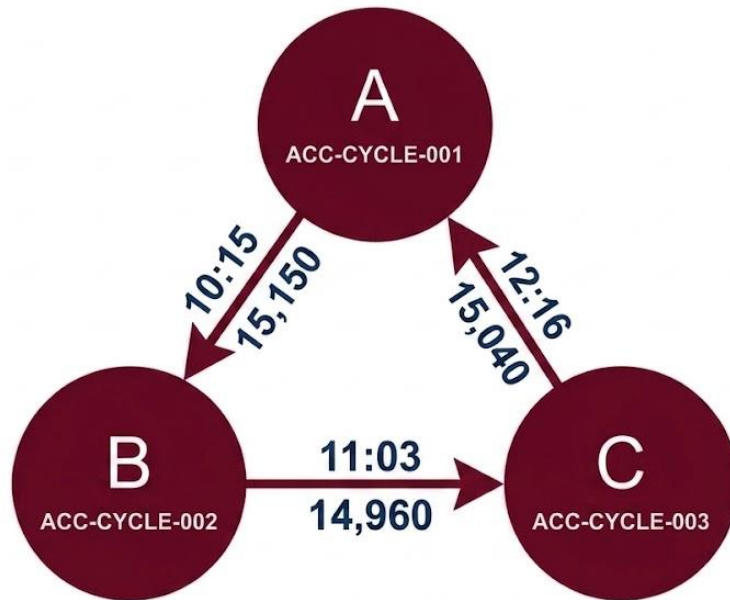
Sort Key: structured_total DESC, tx_time
rows=9 Execution Time ≈ 2.503 ms

What the database does internally

Same rule on aml_graph."TRANSFER" edges, but every **amount/tx_time** is unwrapped via **agtype_access_operator** and **cast from agtype → Postgres type** on every row. After grouping deposits by beneficiary vertex, a **Hash Join** back against aml_graph."Account" binds the beneficiary **before the outbound TRANSFER lookup**, then **three more Nested Loop joins** with public.accounts + public.customers recover typed PK values. Extra casts + vertex bindings push it to **≈ 2.503 ms / 175 shared buffers**

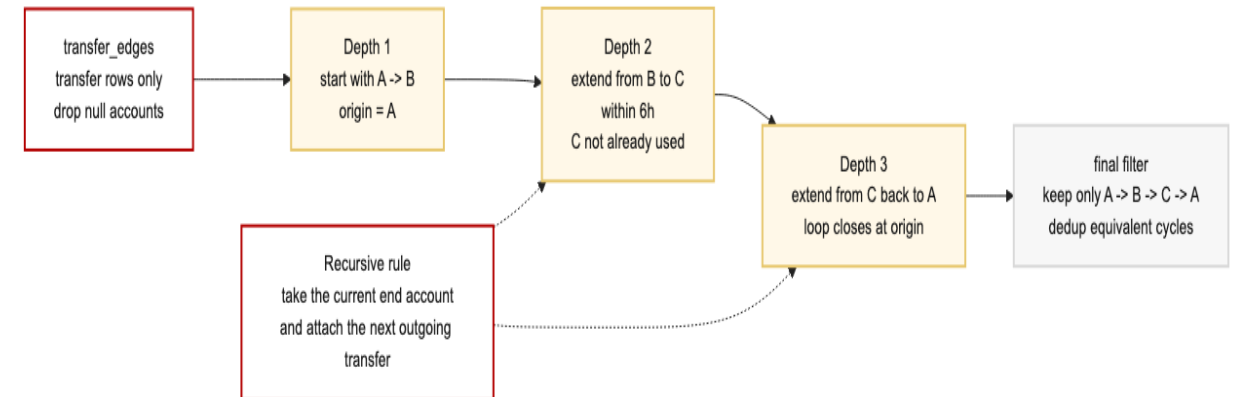
What the application does 2: Circular trading detection

Circular Trading Scenario Visual Explanation



Three distinct accounts complete an ordered A → B → C → A loop within 6 hours.

SQL Logic

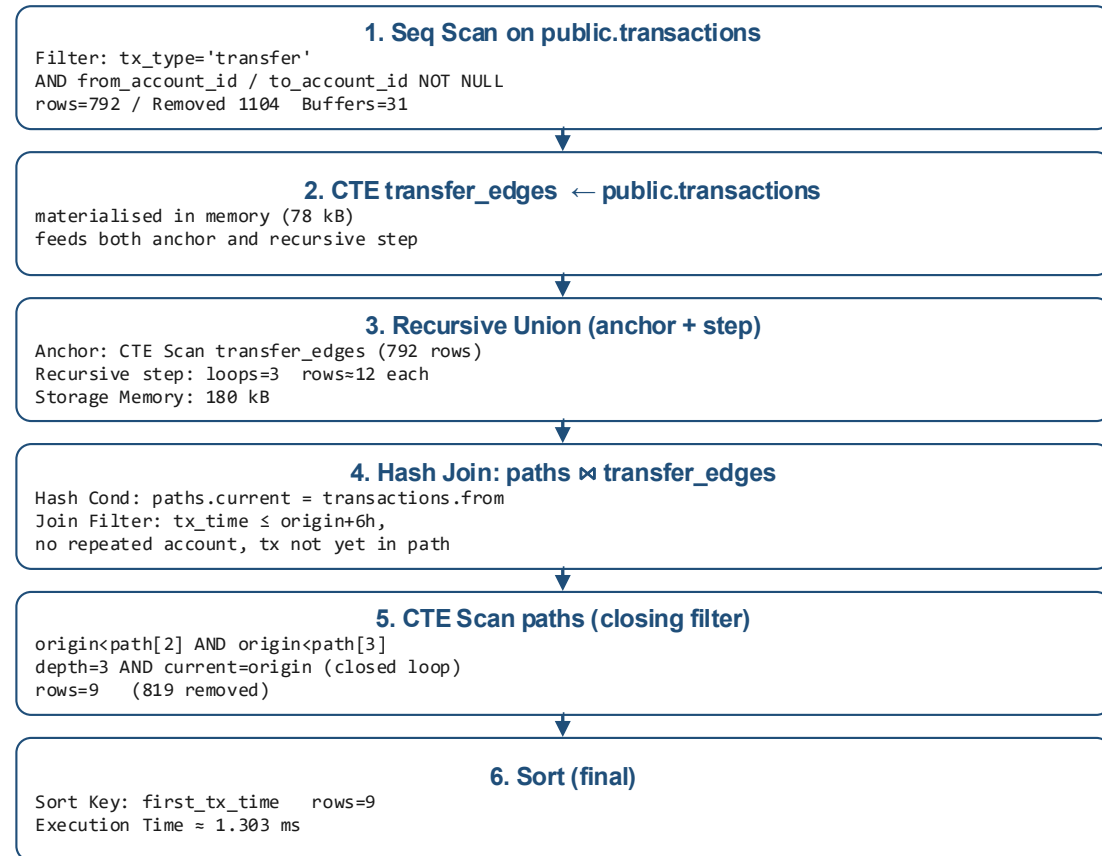


Real-World Example

Several transactions take place among three different accounts in a short period of time. First, account **A (ACC-CYCLE-001)** sends \$15,150 to account **B (ACC-CYCLE-002)** at 10:15. Then, account **B** sends \$14,960 to account **C (ACC-CYCLE-003)** at 11:03. Finally, account **C** sends \$15,040 back to account **A** at 12:16. This creates a closed loop of money flow, **A → B → C → A**, within only a few hours. This pattern is suspicious because the funds circulate among related accounts and return close to the starting point, which may indicate **circular trading or layering behavior used to create artificial transaction activity and obscure the true origin of funds**.

Query Explanation 2: Circular trading EXPLAIN analysis

PostgreSQL SQL Execution EXPLAIN



What the database does internally

Filter public.transactions → transfers, materialise as **CTE transfer_edges** (78 kB). A **Recursive Union** grows 3-hop paths: anchor reads transfer_edges directly, each recursive step does a **Hash Join** on another transfer_edges row. **Join filter** enforces a **6-hour window**, **no repeated account**, and **no re-used transaction**. A **closing CTE scan** keeps only **A → B → C → A** loops, then a **final Sort** by first_tx_time. Result: **9 cycles in ≈ 1.303 ms / 31 shared buffers**.

APACHE AGE Execution EXPLAIN



Explanation

Three separate scans of aml_graph."TRANSFER" (one per hop) stitched by **Hash Joins + Nested Loops** on aml_graph."Account" vertices. **Index Scan** on TRANSFER_start_id_idx dominates: **loops=792** (hop 1), **loops=477** (hop 2). **Edge uniqueness** (_ag_enforce_edge_uniqueness3) and **tx_time ordering** (hop1 ≤ hop2 ≤ hop3) enforced as **Join Filters** on top of every agtype property access. Result: **> 10,600 shared buffers, ≈ 6.97 ms — about 5× slower** than the recursive SQL.

Why it matters: tuned graph narrows traversal, but SQL still executes less work on this dataset.

Focus Area 2: Query Execution

How we measured query performance



Smurfing Detection

PostgreSQL

0.443 ms

median exec time

Apache Age

2.503 ms

median exec time

Circular Trading Detection

PostgreSQL

1.303 ms

median exec time

Apache Age

6.97 ms

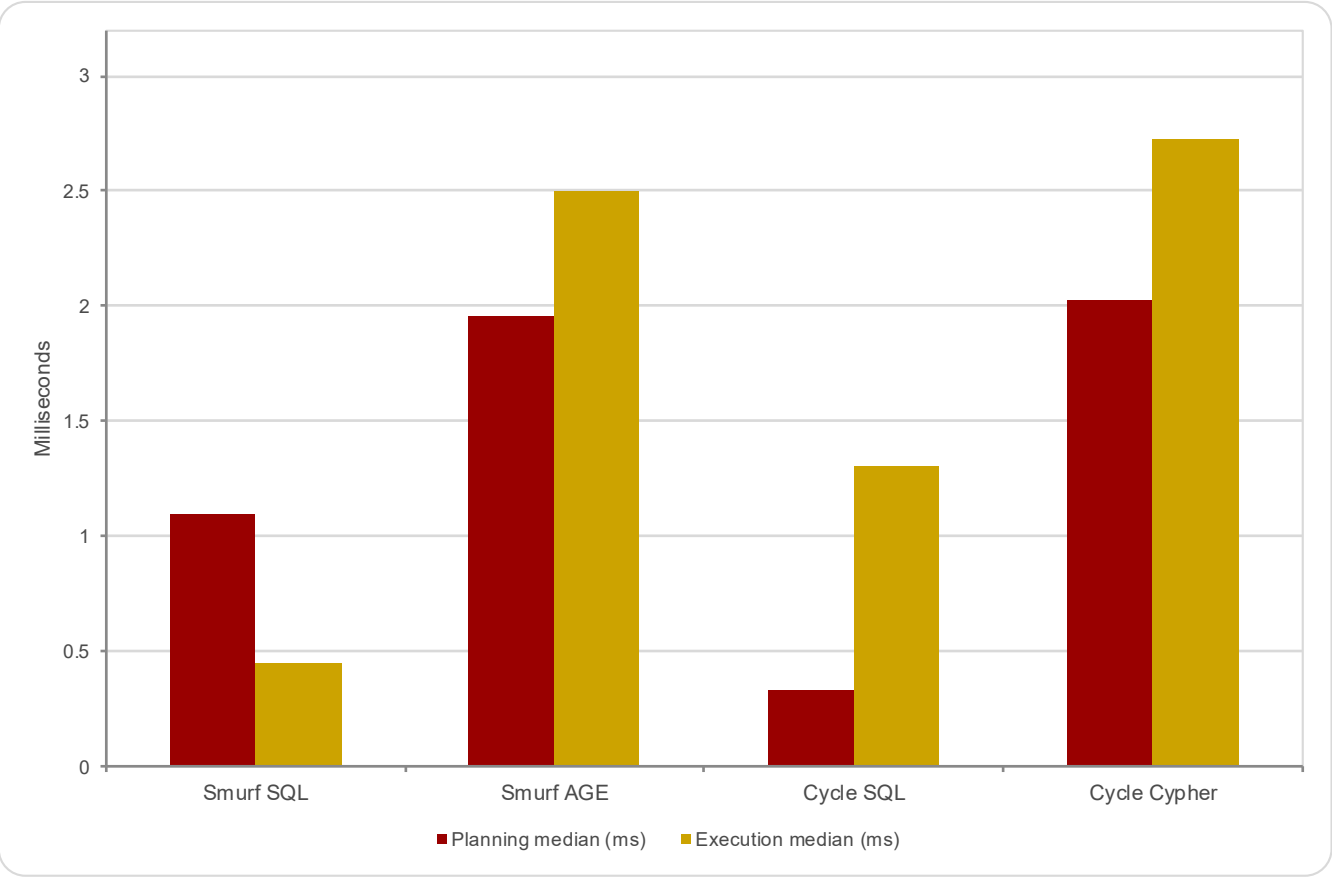
median exec time

Why it matters (Exec time performance Analysis)

- Every property access on an AGE edge or vertex goes through agtype_access_operator and is then cast from agtype to the underlying Postgres type. That happens on every amount, tx_time, account_id, and tx_id — once per row, per join condition.
- The graph layer also forces extra vertex joins the relational plan never needs: a Hash Join on aml_graph."Account" to resolve the beneficiary before the outbound transfer (smurfing), and full TRANSFER + Account re-scans for every hop (cycle detection). _ag_enforce_edge_uniqueness adds a final filter on each edge.
- Buffer usage shows the impact clearly: smurfing touches 175 shared buffers on AGE versus 169 on SQL, and circular-trading touches roughly 10,648 buffers on AGE versus only 31 on the recursive SQL plan. The end result is $\approx 5.7\times$ slower for smurfing and $\approx 5\times$ slower for cycle detection, even though the Cypher is noticeably shorter to write.

Evaluation, closing, and future scope

Median planning and execution times summarize the current answer to the research question.



Feasibility

One PostgreSQL stack can support both relational AML analysis and graph-style AML analysis on the same dataset.

Performance takeaway

SQL is faster on the current workload, especially for aggregation-heavy smurfing, because it avoids graph-layer property extraction.

Future scope

Fine-tune the graph further, enlarge the workload, and stress more traversal-heavy AML rules where the graph projection may matter more.

Measured takeaway: one-stack feasibility is proven, SQL is faster on the current workload, and future work is more graph tuning plus larger workloads.

0.443 ms smurf SQL exec	2.503 ms smurf AGE exec	1.303 ms cycle SQL exec	6.97 ms cycle Cypher exec
----------------------------	----------------------------	----------------------------	------------------------------

Appendix A1: Live demo runbook

Step	Live action	Expected confirmation
1	State the research question and why graph analysis matters in AML.	Audience knows the demo is about one-stack feasibility plus internals.
2	Run ./scripts/reset_pipeline.sh.	Schema, data load, and AGE projection finish successfully.
3	Execute smurfing SQL.	9 detections appear from the relational query path.
4	Execute smurfing AGE.	The same 9 detections appear from the Cypher path.
5	Execute circular recursive SQL and tuned Cypher.	Both loop queries return 9 detections.
6	Close on EXPLAIN medians and focus areas.	Storage Architecture and Query Execution explain the runtime gap.

REPRESENTATIVE COMMANDS

```
./scripts/reset_pipeline.sh
docker exec -i -w /workspace apache-age \
  psql -U postgres -d hybrid_aml \
    < database/queries/smurfing_detection.sql
docker exec -i -w /workspace apache-age \
  <
  database/queries/smurfing_detection_age.sql
docker exec -i -w /workspace apache-age \
  <
  database/queries/circular_trading_recursive
  .sql
docker exec -i -w /workspace apache-age \
  <
  database/queries/circular_trading_cypher_tu
  ned.sql
```

Presenter note

If time is tight, run reset first and then show one SQL path plus one tuned Cypher path before returning to the evaluation slide.

Appendix A2: TA / instructor runnable checklist

Requirement	Evidence	How satisfied
Application source code	database/, src/, docs/demo/	The repo contains the SQL, Cypher, generation logic, and deck source used in the demo.
Database schema/setup scripts	001_schema.sql, setup_age.sql, load_age_graph*.sql	A reviewer can recreate the relational schema and both graph projections.
Instructions to run	README.md and reset_pipeline.sh	The run order is documented and repeatable from the repo root.
Small dataset	data/synthetic/*.csv	The dataset is compact enough for classroom reruns and labeled enough for correctness checks.
Runnable demo queries	smurfing_detection*.sql and circular_trading*.sql	Each major operation has executable queries and matching EXPLAIN evidence.

Appendix A3: Backup evidence

Docker + PostgreSQL + AGE

```
$ docker ps
NAME          IMAGE          STATUS          PORTS
apache-age    apache/age     Up 56 minutes   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

$ psql -d hybrid_aml -c 'SELECT version();'
version
-----
PostgreSQL 18.1 (Debian 18.1-1.pgdgl3+2) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19...
(1 row)

$ psql -d hybrid_aml -c "SELECT extname, extversion FROM pg_extension WHERE extname = 'age';"
extname | extversion
-----+-----
age     | 1.7.0
(1 row)

$ psql -d hybrid_aml -c 'SELECT COUNT(*) FROM customers/accounts/transactions;'
customers
-----
33
(1 row)

accounts
-----
38
(1 row)

transactions
-----
209
(1 row)
```

Apache AGE Graph Projection

```
$ psql -d hybrid_aml -f sql/load_age_graph.sql
LOAD
SET
graphid | name | namespace
-----+-----+-----
18555 | aml_graph | aml_graph
(1 row)

name | kind | relation
-----+-----+-----
_ag_label_vertex | v | aml_graph._ag_label_vertex
_ag_label_edge | e | aml_graph._ag_label_edge
Account | v | aml_graph."Account"
TRANSFER | e | aml_graph."TRANSFER"
(4 rows)

$ psql -d hybrid_aml -f sql/age_smurfing_subgraph.sql
LOAD
SET
from_account | tx_id | amount | channel | tx_time | to_account
-----+-----+-----+-----+-----+-----
"ACC-FEED-001" | "TX-00202" | 8940.0 | "cash" | "2026-02-18 09:05:00" | "ACC-AML-HUB-001"
"ACC-FEED-002" | "TX-00203" | 9180.0 | "cash" | "2026-02-18 09:32:00" | "ACC-AML-HUB-001"
"ACC-FEED-003" | "TX-00204" | 9420.0 | "cash" | "2026-02-18 10:04:00" | "ACC-AML-HUB-001"
"ACC-FEED-004" | "TX-00205" | 9760.0 | "cash" | "2026-02-18 10:41:00" | "ACC-AML-HUB-001"
"ACC-FEED-005" | "TX-00206" | 9860.0 | "cash" | "2026-02-18 11:18:00" | "ACC-AML-HUB-001"
"ACC-FEED-006" | "TX-00207" | 9640.0 | "cash" | "2026-02-18 11:54:00" | "ACC-AML-HUB-001"
"ACC-AML-HUB-001" | "TX-00208" | 54900.0 | "wire" | "2026-02-18 13:10:00" | "ACC-SHELL-001"
(7 rows)
```

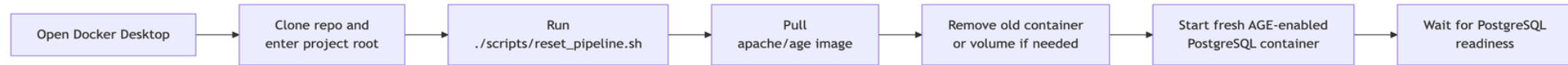
Hybrid-AML Shell Setup Workflow

Docker Desktop + reset_pipeline.sh + PostgreSQL + Apache AGE

1. Reset and Start Environment

Bring up a clean AGE-enabled PostgreSQL container from the repository root.

Relevant files: `scripts/reset_pipeline.sh` `README.md`



2. Load Data and Build Graphs

Generate synthetic CSVs, load relational tables, then project the workload into AGE.

Relevant files: `src/data_generation/generate_synthetic_data.py` `database/migrations/001_schema.sql` `database/seeds/load_from_csv.sql` `database/setup_age.sql`
`database/seeds/load_age_graph.sql` `database/seeds/load_age_graph_tuned.sql`



3. Demo Queries and Benchmark

Show the AML queries in order, then optionally run the performance benchmark.

Relevant files: `database/queries/smurfing_detection.sql` `database/queries/smurfing_detection_age.sql` `database/queries/circular_trading_recursive.sql`
`database/queries/circular_trading_cypher.sql` `database/queries/circular_trading_cypher_tuned.sql` `src/analysis/benchmark_cycle_queries.py`

